Missing Numbers

AlgoExpert · Medium · Arrays

The problem

- Array holds 1..n+2 with two numbers missing; return them ascending.
- Split by the average of the two missing values and sum each half.

Intuition

- The numbers one to n plus two sum to a known total; the missing pair splits around their average.

Brute force vs optimal

- Brute force: check each 1...n for membership → $O(n^2)$
- Hash set → O(n) time, O(n) space
- Math (sums, or XOR) → O(n) time, O(1) space

Approach

- 1. Compute the missing sum
- 2. Split at the average
- 3. Sum the expected and actual values below it
- 4. The difference gives one number, the rest gives the other

Worked example

1...n (grey = present)




```
n = 5
missing sum = 15 - 8 = 7
```

Missing pair sums to 7

Worked example

```
split at average = 7 // 2 = 3

one missing number is ≤ avg, the other > avg
```

Split at 3

Worked example

```
expected sum 1…3 = 6

actual sum of inputs ≤ 3 = 4

first missing = 6 − 4 = 2
```

First missing = 2

Worked example




```
second = 7 - 2 = 5
```

Answer: [2, 5]

Java solution

```java
public class MissingNumbers {
    public static int[] missingNumbers(int[] nums) {
        int n = nums.length + 2;
        long total = (long) n * (n + 1) / 2;
        for (int x : nums) total -= x;
        long avg = total / 2, lowExpected = avg * (avg + 1) / 2, lowActual = 0;
        for (int x : nums) if (x <= avg) lowActual += x;
        int a = (int) (lowExpected - lowActual);
        return new int[]{a, (int) (total - a)};
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Use long for big sums
- Missing numbers may be 1 or n
- Empty input → [1, 2]

Interview tips & follow-ups

- Mention the XOR variant
- Follow-up: k missing numbers → cyclic sort / bitset

Recap

- The numbers one to n plus two sum to a known total; the missing pair splits around their average.
- Complexity: O(n)